

TP 1 et 2 : Middleware pour l'IoT

I. Introduction

L'Internet des Objets (IoT) est caractérisée par l'interconnexion massive d'objets physiques au réseau. Dans le cadre d'une maison intelligente, des dizaines de dispositifs connectés génèrent et échangent des données en continu. La gestion de ces périphériques, souvent hétérogènes et limités en ressources énergétiques et en puissance de calcul, nécessite des protocoles de communication particulièrement optimisés, fiables et légers pour garantir une transmission fluide des informations.

L'objectif de ce TP est de découvrir le protocole MQTT (Message Queuing Telemetry Transport), aujourd'hui un standard pour la communication IoT. Pour cela, une application simple, basée sur un ESP8266, sera développée pour montrer ses fonctionnalités et son fonctionnement.

II. État de l'art : architecture IoT et Middleware

Avant d'aborder le protocole MQTT, il est nécessaire de comprendre l'architecture globale dans laquelle il s'inscrit, et notamment le rôle du Middleware. Le Middleware est une couche logicielle essentielle qui se positionne entre le système d'exploitation et la couche applicative. Il s'agit d'un logiciel tournant sur l'OS dont le rôle est d'aider les autres applications à interagir ensemble ou avec le système d'exploitation lui-même. Il permet ainsi au développeur de se concentrer sur la logique métier de son application sans avoir à se soucier des contraintes extérieures, telles que la gestion bas niveau du matériel.

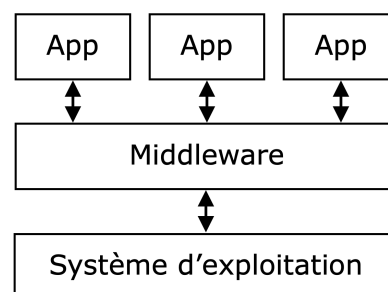


Figure 1 : Rôle du Middleware.

Une architecture IoT classique se décompose généralement en quatre niveaux :

- les dispositifs (capteurs, cartes embarquées, passerelles),
- les réseaux de communication (Wi-Fi, BLE, LoRaWAN, NB-IoT, LTE-M, etc.),
- le Middleware IoT,
- les applications, dashboards ou algorithmes.

Dans cette structure, le Middleware fournit plusieurs fonctionnalités. Il assure une abstraction matérielle permettant de changer de capteur sans réécrire toute l'application et offre une gestion unifiée des protocoles (MQTT, CoAP, HTTP, Modbus ou OPC-UA). Il propose également des services transversaux (comme la sécurité, le stockage ou la normalisation des données), et expose des APIs pour les développeurs. Parmi les

exemples notables de middleware, on peut citer ThingsBoard ou la suite Eclipse IoT comprenant Mosquitto, Ditto, Leshan/Lwm2m et Kura.

La notion d'interopérabilité est également centrale. Elle désigne la capacité de deux systèmes à fonctionner ensemble en échangeant des données ou des services. Lorsque deux systèmes ne sont pas nativement compatibles, un intermédiaire est nécessaire pour servir de traducteur. C'est dans ce cadre qu'intervient oneM2M, une norme internationale définissant une architecture unifiée pour les systèmes IoT.

III. Réponses aux questions

Question 1 : Quelle est l'architecture typique d'un système IoT basé sur le protocole MQTT ?

L'architecture d'un système IoT basé sur MQTT suit un modèle centralisé de type étoile, ou star topology, avec un broker (serveur) placé au centre du réseau. On y trouve donc :

- Des clients (Publishers/Subscribers) : Ce sont les appareils IoT (capteurs, actionneurs, microcontrôleurs) ou les applications de contrôle (smartphones, serveurs). Un client peut être à la fois publisher (pour envoyer des données de capteur) et subscriber (pour recevoir des commandes)
- Un broker : C'est le point central de l'architecture. Il reçoit tous les messages des publishers sur des topics spécifiques. Il filtre ces messages et les envoie aux subscribers qui se sont abonnés à ces topics. Il assure le découplage temporel et spatial entre l'émetteur et le récepteur : ils n'ont pas besoin d'être connectés en même temps ni de connaître l'adresse IP de l'autre.

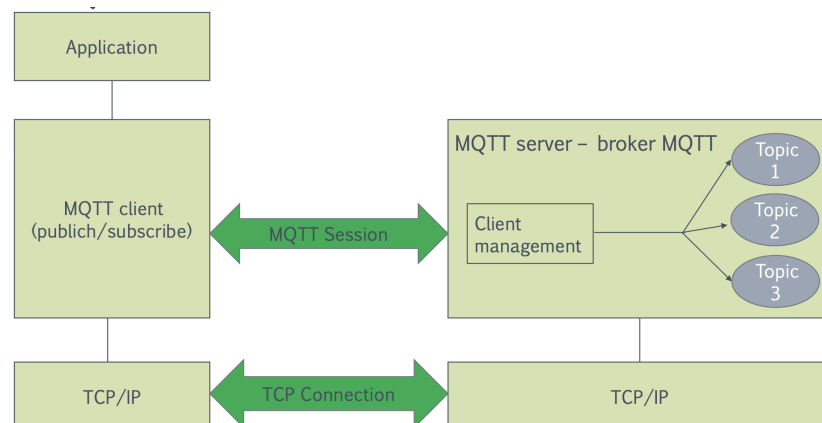


Figure 2 : Architecture d'un système IoT basé sur MQTT.

Question 2 : Quel est le protocole IP sous-jacent à MQTT ? Qu'est-ce que cela signifie en termes de bande passante, type de communication, etc. ?)

MQTT utilise TCP/IP (Transmission Control Protocol/Internet Protocol). L'utilisation de TCP garantit une fiabilité importante car les messages sont livrés dans l'ordre et sans perte, contrairement au protocole UDP.

TCP est un protocole avec connexion, ce qui augmente l'utilisation de la bande passante à cause de la nécessité d'envoi de paquet de synchronisation et de paquet d'acquiescement. Cependant, une fois la connexion MQTT établie, elle reste ouverte longtemps. Ainsi, le coût du handshake est amorti et les acquiescements deviennent légers par rapport au volume utile, ce qui permet au trafic MQTT de rester très peu gourmand en bande passante.

La communication est par ailleurs persistante (un canal TCP reste ouvert entre le client et le broker) et découplée (contrairement au modèle HTTP classique). Le publisher envoie des messages au broker sans se soucier des abonnés potentiels, et le subscriber reçoit des messages du broker sans avoir besoin de connaître l'identité de l'émetteur.

Question 3 : Quelles sont les différentes versions de MQTT ?

Il existe trois principales versions du protocole MQTT :

- MQTT 3.1 (2010), qui est la première version largement utilisée, permet d'utiliser le modèle publish/subscribe et les topics hiérarchiques.
- MQTT 3.1.1 (2014), qui est la version la plus répandue aujourd'hui dans l'IoT classique. Elle présente plusieurs améliorations, corrige des ambiguïtés et simplifie des règles. Cette version ajoute aussi le « protocol name » fixe (« MQTT ») qui est une chaîne de texte incluse au début d'une connexion et qui sert à identifier clairement le protocole utilisé. Avant cette version, plusieurs variantes existaient en fonction des opérateurs (« MQIsdp » par exemple). C'est la version supportée par presque tous les brokers (dont Mosquitto).
- MQTT 5.0 (2019), qui est une version moderne, plus puissante. Elle n'apporte pas de rupture, mais ajoute des fonctionnalités avancées, comme des métadonnées, des codes d'erreur, une réduction supplémentaire de la bande passante, etc.

Question 4 : Quels types de sécurité / authentification / chiffrement sont utilisés dans MQTT ?

Concernant le chiffrement, MQTT ne l'inclut pas nativement, ce qui implique que les données sont en texte clair par défaut. Il est donc presque toujours encapsulé dans TLS/SSL (Transport Layer Security), généralement sur le port 8883, pour assurer un chiffrement de bout en bout.

Pour l'authentification, le client transmet un nom d'utilisateur et un mot de passe lors de la connexion, que le broker vérifie via une base de données ou un fichier de configuration. Une méthode plus robuste consiste à utiliser des certificats TLS où les clients et/ou le broker doivent présenter un certificat numérique valide pour établir la connexion.

Enfin, l'autorisation est gérée une fois le client authentifié : le broker utilise des listes de contrôle d'accès (ACLs, Access Control Lists) pour déterminer précisément quels utilisateurs ont le droit de publier ou de s'abonner à quels topics.

Question 5 : Supposons que vous ayez des appareils comprenant un bouton, une lampe et un capteur de luminosité. Vous souhaitez créer un système intelligent pour votre maison avec ce comportement : pouvoir allumer la lumière manuellement avec le bouton et que la lumière s'allume automatiquement lorsque la luminosité est inférieure à une certaine valeur. Quels sont les différents topics nécessaires et quelles seront les connexions en termes de publication ou d'abonnement ?

Le capteur lit en continu les valeurs de luminosité et les publie sur le topic *maison/salon/luminosite/mesure*.

Lorsque l'utilisateur appuie sur le bouton. L'appareil bouton publie 1 sur *maison/salon/bouton/etat*.

Un orchestrateur est abonné aux deux topics de mesure. Dès qu'il reçoit un nouveau message sur l'un ou l'autre, il applique la règle logique : SI (luminosité < seuil) OU (bouton = pressé) ALORS état = ON. Il publie l'ordre final (ON ou OFF) sur le topic *maison/salon/lampe/commande*.

La lampe est abonnée à *maison/salon/lampe/commande*. Elle reçoit l'ordre (ON ou OFF) et exécute l'action physique (allumer ou éteindre la lumière).

Le schéma suivant modélise la communication.

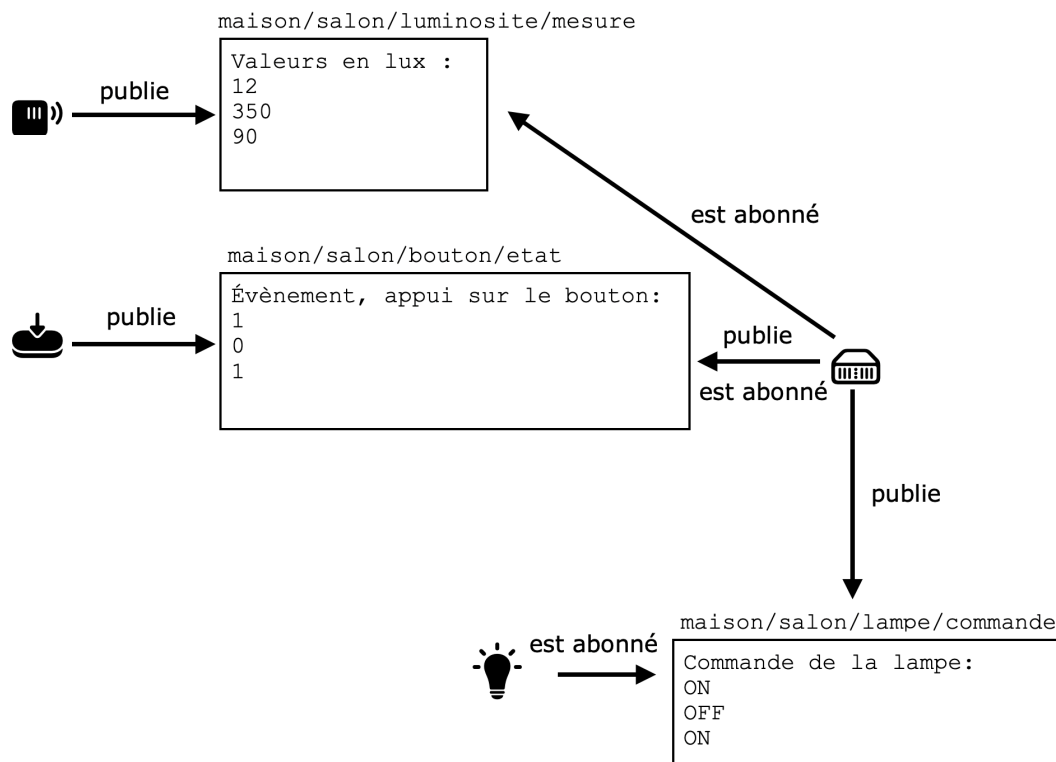
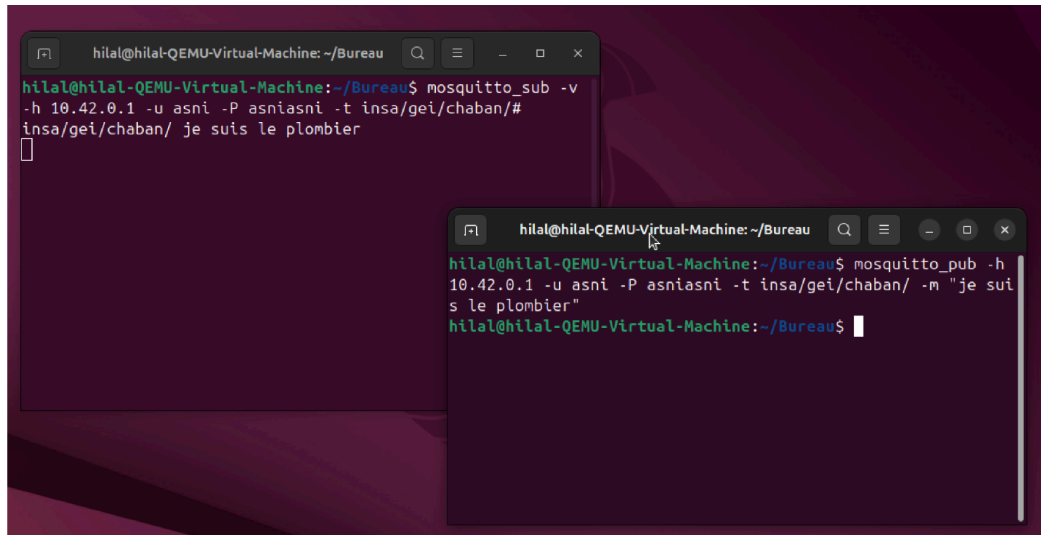


Figure 3 : Communication du système.

IV. Installation et test du Broker MQTT (Mosquitto)

Mosquitto est une implémentation open-source du protocole MQTT. Il fonctionne comme un serveur central qui achemine les messages entre les différents clients (publishers et subscribers). Une fois le broker démarré, la commande `mosquitto_sub` a été utilisée dans un premier terminal pour établir un client abonné au topic `insa/gei/chaban`, attendant de recevoir des messages. Simultanément, la commande `mosquitto_pub` a été exécutée dans un second terminal pour publier un message de test, ici le texte "je suis le plombier", sur ce même topic.



```

hilar@hilar-QEMU-Virtual-Machine: ~/Bureau
hilar@hilar-QEMU-Virtual-Machine:~/Bureau$ mosquitto_sub -v
-h 10.42.0.1 -u asni -P asniasni -t insa/gei/chaban/#
insa/gei/chaban/ je suis le plombier

hilar@hilar-QEMU-Virtual-Machine: ~/Bureau
hilar@hilar-QEMU-Virtual-Machine:~/Bureau$ mosquitto_pub -h
10.42.0.1 -u asni -P asniasni -t insa/gei/chaban/ -m "je suis le plombier"
hilar@hilar-QEMU-Virtual-Machine:~/Bureau$

```

Figure 4 : Test de Mosquitto avec `mosquitto_sub` et `mosquitto_pub`.

V. Création d'un dispositif IoT avec la carte NodeMCU utilisant la communication MQTT

La carte NodeMCU, construite autour du module ESP-12E (intégrant la puce ESP8266), présente les caractéristiques principales suivantes :

Elle intègre un module WiFi (802.11 b/g/n). Elle permet donc de se connecter à un réseau sans fil en tant que client (station) ou de fonctionner comme un point d'accès.

Elle supporte TCP/IP et UDP. Elle est donc largement utilisée pour les protocoles IoT comme MQTT (mais aussi HTTP).

Elle offre plusieurs GPIO utilisables pour des entrées ou sorties numériques, quelques broches capables de PWM, ainsi qu'une unique entrée analogique limitée à 1 V. La carte dispose aussi d'interfaces série UART, I2C et SPI, ce qui permet de dialoguer avec de nombreux capteurs ou périphériques.

Elle est alimentée en 3,3 V et possède un convertisseur USB-série.

Elle se programme principalement en Lua via le firmware NodeMCU mais plus souvent en C/C++ avec l'IDE Arduino

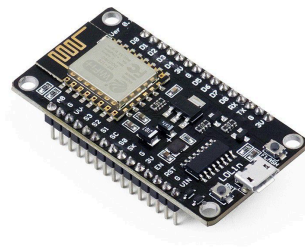


Figure 5 : Carte NodeMCU utilisée.

Le code-exemple (voir Annexe 1) gère la connectivité et la communication IoT. Après avoir inclus les bibliothèques ESP8266WiFi, MQTTClient et ArduinoJson, le programme commence par établir la connexion au réseau sans fil local via l'identifiant (SSID) et le mot de passe configurés. Une fois connecté au WiFi, il initialise le client MQTT en spécifiant l'adresse IP du broker Mosquitto (10.42.0.1), le port 1883, et les identifiants de connexion (asni/asniasni). Le dispositif est ensuite configuré pour s'abonner au topic *insa/gei/chaban* via la fonction `connectToMQTT()`. La boucle principale du programme (`loop()`) permet le maintien permanent de cette connexion MQTT et gère l'envoi périodique d'un message JSON (contenant un horodatage et une valeur analogique lue sur A0) vers le même topic (*insa/gei/chaban*) à intervalles réguliers (5 secondes). Parallèlement, une fonction de callback (`messageHandler`) est définie pour traiter et afficher sur le moniteur série les messages reçus sur le topic.

Le code a été téléversé sur la carte NodeMCU. Pour valider la publication, un client Mosquitto a été utilisé en ligne de commande (`mosquitto_sub -v -h 10.42.0.1 -u asni -P asniasni -t \#`) pour s'abonner au même topic (*insa/gei/chaban*). Les messages publiés par la NodeMCU, contenant le timestamp et la valeur data (analogue à `analogRead(0)`), sont bien reçus par le client abonné.

```

hilar@hilar-QEMU-Virtual-Machine: ~/Bureau
insa/gei/outTopic Groupe Diani Benckekroun #117
insa/gei/outTopic Groupe Diani Benckekroun #118
insa/gei/outTopic Groupe Diani Benckekroun #119
insa/gei/outTopic Groupe Diani Benckekroun #120
insa/gei/outTopic Groupe Diani Benckekroun #121
insa/gei/outTopic Groupe Diani Benckekroun #122
insa/gei/outTopic Groupe Diani Benckekroun #123
insa/gei/outTopic Groupe Diani Benckekroun #124
insa/gei/outTopic Groupe Diani Benckekroun #125
insa/gei/outTopic Groupe Diani Benckekroun #126
insa/gei/outTopic Groupe Diani Benckekroun #127
insa/gei/outTopic Groupe Diani Benckekroun #128
insa/gei/chaban {"timestamp":6466,"data":3}
insa/gei/chaban {"timestamp":11561,"data":3}
insa/gei/chaban {"timestamp":16563,"data":3}
insa/gei/chaban {"timestamp":21565,"data":2}

```

Figure 6 : Messages reçus.

VI. Création d'une application simple

Le code est disponible en Annexe 2.

Toutes les 500 ms, l'ESP8266 lit la valeur analogique du capteur de luminosité et publie cette valeur sur le topic *insa/gei/chaban/capteur*. Il lit aussi l'état du bouton et le publie sur *insa/gei/chaban/bouton* uniquement si cet état a changé.

Lorsqu'un message arrive sur *insa/gei/chaban/capteur*, la valeur est mise à jour dans la variable locale *capteur_value*.

Lorsqu'un message arrive sur *insa/gei/chaban/bouton*, l'état est mis à jour dans la variable locale *bouton_state*.

La fonction *changeLedState* est celle qui "donne l'ordre" à la LED de s'allumer ou de s'éteindre. Elle s'allume si l'état du bouton est 1 ou si la valeur du capteur est faible (<120). Elle s'éteint si l'état du bouton est 0 et si la valeur du capteur est élevée (≥ 120). Après avoir changé l'état de la LED, l'ESP8266 publie le nouvel état de la LED sur le topic *insa/gei/chaban/lampadaire* pour informer le reste du système.

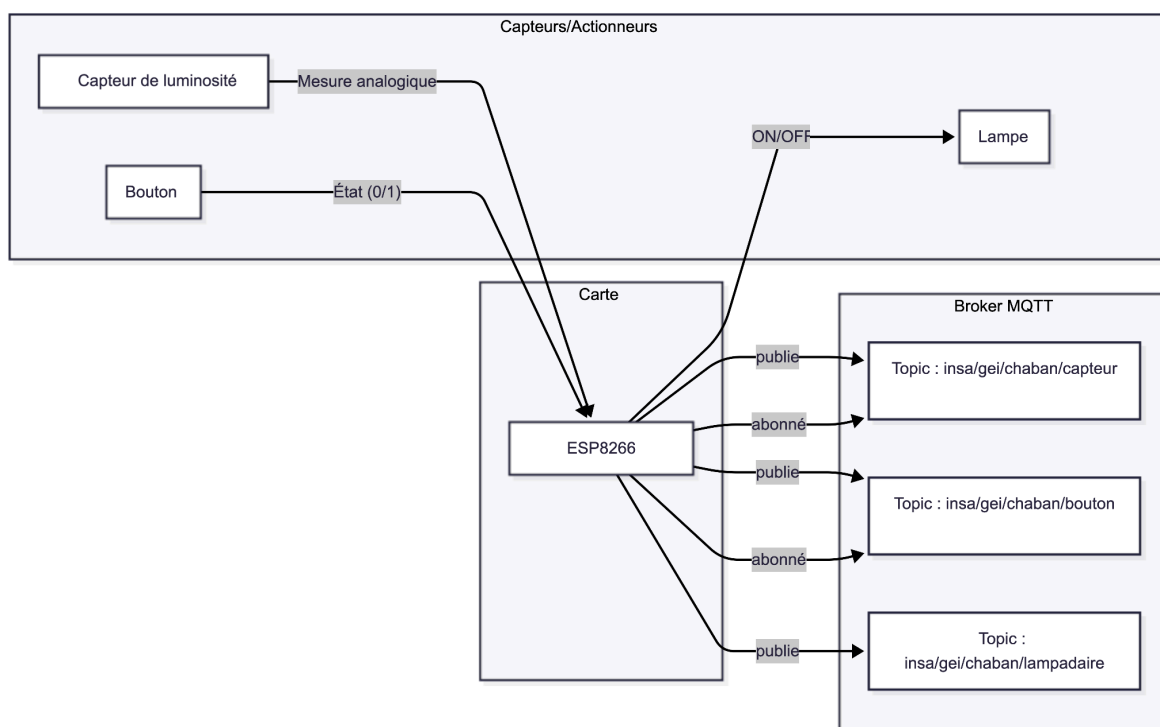


Figure 7 : Architecture de communication.

Les messages reçus, témoignant du bon fonctionnement du système, sont montrées dans les captures d'écran suivantes.

```

hilal@hilal-QEMU-Virtual-Machine: ~/Bureau
hilal@hilal-QEMU-Virtual-Machine:~/Bureau$ mosquitto_sub -v -h 10.42.0.1 -u asni
-P asniasni -t insa/gei/chaban/#
insa/gei/chaban/capteur 741
insa/gei/chaban/capteur 741
insa/gei/chaban/capteur 741
insa/gei/chaban/capteur 741
insa/gei/chaban/capteur 741
insa/gei/chaban/capteur 741
insa/gei/chaban/capteur 741
Valeurs de luminosité > 270
insa/gei/chaban/capteur 741
insa/gei/chaban/capteur 741
insa/gei/chaban/capteur 741
insa/gei/chaban/capteur 741
insa/gei/chaban/capteur 276
insa/gei/chaban/capteur 55
Valeurs de luminosité < 270
insa/gei/chaban/lampadaire 1 Allumage de la lampe
insa/gei/chaban/capteur 12
insa/gei/chaban/capteur 12
insa/gei/chaban/capteur 12
Valeurs de luminosité < 270
insa/gei/chaban/capteur 14
insa/gei/chaban/capteur 15
insa/gei/chaban/capteur 13

```

Figure 8 : Allumage de la lampe en cas de faible luminosité.

```

hilal@hilal-QEMU-Virtual-Machine: ~/Bureau
hilal@hilal-QEMU-Virtual-Machine:~/Bureau$ mosquitto_sub -v -h 10.42.0.1 -u asni
-P asniasni -t insa/gei/chaban/#
insa/gei/chaban/capteur 740
insa/gei/chaban/capteur 740
insa/gei/chaban/capteur 740
Valeurs de luminosité
insa/gei/chaban/capteur 740
insa/gei/chaban/capteur 740
insa/gei/chaban/bouton 1 Appui sur le bouton
insa/gei/chaban/lampadaire 1 Allumage de la lampe
insa/gei/chaban/bouton 0 Bouton relâché
insa/gei/chaban/lampadaire 0 La lampe s'éteint
insa/gei/chaban/bouton 1
insa/gei/chaban/lampadaire 1
insa/gei/chaban/capteur 740
insa/gei/chaban/bouton 0
insa/gei/chaban/lampadaire 0
insa/gei/chaban/bouton 1
insa/gei/chaban/lampadaire 1
insa/gei/chaban/bouton 0
insa/gei/chaban/lampadaire 0
insa/gei/chaban/bouton 1
insa/gei/chaban/lampadaire 1
insa/gei/chaban/capteur 740
insa/gei/chaban/bouton 0

```

Figure 9 : Test du bouton.

Une vidéo montrant le fonctionnement du système est disponible ici : https://drive.google.com/drive/folders/14f1HEHZ3nct4mJdK-IM-AiAe979oqV_K?usp=share_link.



VII. Conclusion

Ce TP a permis de comprendre le rôle du Middleware dans l'architecture IoT, agissant comme une couche d'abstraction pour gérer l'hétérogénéité des dispositifs. Nous avons ensuite exploré en détail le protocole MQTT (Message Queuing Telemetry Transport), standard de communication léger, fiable et optimisé pour les ressources limitées. L'étude a confirmé son architecture en étoile, centrée sur le Broker, garantissant un découplage efficace des clients Publishers et Subscribers via des topics.

L'implémentation pratique, utilisant la carte NodeMCU (ESP8266) et le broker Mosquitto, a concrétisé ces concepts. Nous avons validé la communication publish/subscribe, d'abord avec un simple échange, puis en réalisant une application de maison intelligente fonctionnelle.



Figure Historique 1 : Jacques Chaban Delmas, résistant et ancien Premier Ministre français, dont le nom a été utilisé pour nommer les différents topics.

VIII. Annexe 1 : Code-exemple modifié

```

/*
 * This ESP8266 NodeMCU code was developed by newbiely.com
 *
 * This ESP8266 NodeMCU code is made available for public use without
any restriction
 *
 * For comprehensive instructions and wiring diagrams, please visit:
 * https://newbiely.com/tutorials/esp8266/esp8266-mqtt
 */

#include <ESP8266WiFi.h>
#include <MQTTClient.h>
#include <ArduinoJson.h>

const char WIFI_SSID[] = "asni";      // CHANGE TO YOUR WIFI SSID
const char WIFI_PASSWORD[] = "asniasni"; // CHANGE TO YOUR WIFI
PASSWORD

const char MQTT_BROKER_ADDRESS[] = "10.42.0.1"; // CHANGE TO MQTT
BROKER'S ADDRESS
const int MQTT_PORT = 1883;
const char MQTT_CLIENT_ID[] = "chaban-esp8266-07-03-1915"; // CHANGE
IT AS YOU DESIRE
const char MQTT_USERNAME[] = "asni"; // CHANGE
IT IF REQUIRED, empty if not required
const char MQTT_PASSWORD[] = "asniasni"; //
CHANGE IT IF REQUIRED, empty if not required

// The MQTT topics that ESP8266 should publish/subscribe
const char PUBLISH_TOPIC[] = "insa/gei/chaban"; // CHANGE IT AS YOU
DESIRE
const char SUBSCRIBE_TOPIC[] = "insa/gei/chaban"; // CHANGE IT AS YOU
DESIRE

const int PUBLISH_INTERVAL = 5000; // 5 seconds

WiFiClient network;
MQTTClient mqtt = MQTTClient(256);

unsigned long lastPublishTime = 0;

```

```

void setup() {
    Serial.begin(9600);

    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    Serial.println("ESP8266 - Connecting to Wi-Fi");
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }

    Serial.println("");
    Serial.print("Connected to WiFi network with IP Address: ");
    Serial.println(WiFi.localIP());

    connectToMQTT();
}

void loop() {
    mqtt.loop();

    if (millis() - lastPublishTime > PUBLISH_INTERVAL) {
        sendToMQTT();
        lastPublishTime = millis();
    }
}

void connectToMQTT() {
    // Connect to the MQTT broker
    mqtt.begin(MQTT_BROKER_ADDRESS, MQTT_PORT, network);

    // Create a handler for incoming messages
    mqtt.onMessage(messageHandler);

    Serial.print("ESP8266 - Connecting to MQTT broker");

    while (!mqtt.connect(MQTT_CLIENT_ID, MQTT_USERNAME, MQTT_PASSWORD)) {
        Serial.print(".");
        delay(100);
    }
    Serial.println();

    if (!mqtt.connected()) {
        Serial.println("ESP8266 - MQTT broker Timeout!");
    }
}
    
```

```

        return;
    }

    // Subscribe to a topic, the incoming messages are processed by
    messageHandler() function
    if (mqtt.subscribe(SUBSCRIBE_TOPIC))
        Serial.print("ESP8266 - Subscribed to the topic: ");
    else
        Serial.print("ESP8266 - Failed to subscribe to the topic: ");

    Serial.println(SUBSCRIBE_TOPIC);
    Serial.println("ESP8266 - MQTT broker Connected!");
}

void sendToMQTT() {
    StaticJsonDocument<200> message;
    message["timestamp"] = millis();
    message["data"] = analogRead(0); // Or you can read data from other
    sensors
    char messageBuffer[512];
    serializeJson(message, messageBuffer);

    mqtt.publish(PUBLISH_TOPIC, messageBuffer);

    Serial.println("ESP8266 - sent to MQTT:");
    Serial.print("- topic: ");
    Serial.println(PUBLISH_TOPIC);
    Serial.print("- payload:");
    Serial.println(messageBuffer);
}

void messageHandler(String &topic, String &payload) {
    Serial.println("ESP8266 - received from MQTT:");
    Serial.println("- topic: " + topic);
    Serial.println("- payload:");
    Serial.println(payload);
}

```

IX. Annexe 2 : Code du projet

```
#include <ESP8266WiFi.h>
#include <MQTTClient.h>
#include <ArduinoJson.h>
#include <string.h>

#define BUTTON_PIN D5
#define CAPTEUR_PIN A0

const char WIFI_SSID[] = "asni";      // CHANGE TO YOUR WIFI SSID
const char WIFI_PASSWORD[] = "asniasni"; // CHANGE TO YOUR WIFI
PASSWORD

const char MQTT_BROKER_ADDRESS[] = "10.42.0.1"; // CHANGE TO MQTT
BROKER'S ADDRESS
const int MQTT_PORT = 1883;
const char MQTT_CLIENT_ID[] = "chaban-esp8266-07-03-1915"; // CHANGE
IT AS YOU DESIRE
const char MQTT_USERNAME[] = "asni"; // CHANGE
IT IF REQUIRED, empty if not required
const char MQTT_PASSWORD[] = "asniasni"; //
CHANGE IT IF REQUIRED, empty if not required

// The MQTT topics that ESP8266 should publish/subscribe

const char CAPTEUR_TAUPIQUE[] = "insa/gei/chaban/capteur";
const char LAMPADAIRE_TAUPIQUE[] = "insa/gei/chaban/lampadaire";
const char BOUTON_TAUPIQUE[] = "insa/gei/chaban/bouton";

// const char PUBLISH_TOPIC[] = "insa/gei/chaban"; // CHANGE IT AS
YOU DESIRE
// const char SUBSCRIBE_TOPIC[] = "insa/gei/chaban"; // CHANGE IT AS
YOU DESIRE

const int PUBLISH_INTERVAL = 500; // 5 seconds

int led_state = 0;
int capteur_value = 0;
int bouton_state = 0;

WiFiClient network;
MQTTClient mqtt = MQTTClient(256);
```

```

unsigned long lastPublishTime = 0;

void setup() {
    Serial.begin(9600);

    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    Serial.println("ESP8266 - Connecting to Wi-Fi");
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }

    Serial.println("");
    Serial.print("Connected to WiFi network with IP Address: ");
    Serial.println(WiFi.localIP());
    pinMode(LED_BUILTIN, OUTPUT);
    pinMode(BUTTON_PIN, INPUT);
    pinMode(CAPTEUR_PIN, INPUT);

    connectToMQTT();
}

void loop() {
    int current_button_state;
    mqtt.loop();

    if (millis() - lastPublishTime > PUBLISH_INTERVAL) {
        sendToMQTT(CAPTEUR_TAUIQUE, String(analogRead(CAPTEUR_PIN)));
        lastPublishTime = millis();
    }

    current_button_state = digitalRead(BUTTON_PIN);

    if (current_button_state == 1 && bouton_state == 0) sendToMQTT(BOUTON_TAUIQUE,
String(current_button_state));

    if (current_button_state == 0 && bouton_state == 1) sendToMQTT(BOUTON_TAUIQUE,
String(current_button_state));

    changeLedState();
}

void changeLedState() {

```

```

    if(led_state==0 && (bouton_state==1||capteur_value<120)){
        digitalWrite(LED_BUILTIN, LOW);
        led_state=1;
        sendToMQTT(LAMPADAIRE_TAUPIQUE, String(led_state));
    }
    if(led_state==1 && (bouton_state==0 && capteur_value>=120)){
        digitalWrite(LED_BUILTIN, HIGH);
        led_state=0;
        sendToMQTT(LAMPADAIRE_TAUPIQUE, String(led_state));
    }
}

void connectToMQTT() {
    // Connect to the MQTT broker
    mqtt.begin(MQTT_BROKER_ADDRESS, MQTT_PORT, network);

    // Create a handler for incoming messages
    mqtt.onMessage(messageHandler);

    Serial.println("ESP8266 - Connecting to MQTT broker");

    while (!mqtt.connect(MQTT_CLIENT_ID, MQTT_USERNAME, MQTT_PASSWORD)) {
        Serial.print(".");
        delay(100);
    }
    Serial.println();

    if (!mqtt.connected()) {
        Serial.println("ESP8266 - MQTT broker Timeout!");
        return;
    }

    // Subscribe to a topic, the incoming messages are processed by
    messageHandler() function
    if (mqtt.subscribe(CAPTEUR_TAUPIQUE) &&
    mqtt.subscribe(BOUTON_TAUPIQUE))
        Serial.print("ESP8266 - Subscribed to the topic: ");
    else
        Serial.print("ESP8266 - Failed to subscribe to the topic: ");

    Serial.println(CAPTEUR_TAUPIQUE);
    Serial.println(BOUTON_TAUPIQUE);
    Serial.println("ESP8266 - MQTT broker Connected!");
}

```

```

}

void sendToMQTT(String topic, String payload) {
    // StaticJsonDocument<200> message;
    // message["timestamp"] = millis();
    // message["data"] = analogRead(0); // Or you can read data from
other sensors
    // char messageBuffer[512];
    // serializeJson(message, messageBuffer);

    mqtt.publish(topic, payload);

    // Serial.println("ESP8266 - sent to MQTT:");
    // Serial.print("- topic: ");
    // Serial.println(topic);
    // Serial.print("- payload:");
    // Serial.println(payload);
}

void messageHandler(String &topic, String &payload) {
    Serial.println("ESP8266 - received from MQTT:");
    Serial.println("- topic: " + topic);
    Serial.println("- payload:");
    Serial.println(payload);
    if(topic == BOUTON_TAUPIQUE){
        bouton_state = payload.toInt();
    }else if (topic== CAPTEUR_TAUPIQUE){
        capteur_value = payload.toInt();
    }
}

```